

RÉPUBLIQUE DU CAMEROUN
REPUBLIC OF CAMEROON
Peace – Work – Fatherland

UNIVERSITÉ DE DSCHANG
UNIVERSITY OF DSCHANG
Scholae Thesaurus Dschangensis Ibi Cordum

BP 96, Dschang (Cameroun)
Tél./Fax : (237) 233 45 13 81
Website : <http://www.univ-dschang.org>
E-mail : udsrectorat@univ-dschang.org



FACULTÉ DES SCIENCES
FACULTY OF SCIENCES

Département de Mathématiques et
Informatique
*Department of Mathematics and
Computer Science*

BP 67, Dschang (Cameroun)
Tél. : (237) 233 45 17 35
Website : <http://fs.univ-dschang.org>
E-mail : dept-math-info@univ-dschang.org

SUPPORT VECTOR MACHINES, RANDOM FORESTS & EXTREME GRADIENT BOOSTING

Niveau : **MASTER 2**

Option : **Intelligence Artificielle**

Sous la supervision de :

Pr.KENGNE TCHENDJI Vianney
Maître de Conférences, Université de Dschang

Année Académique : 2025 - 2026

Membres du groupe 3

Nom	Matricule
LEKANE KENFACK VERGEZ	CM-UDS-21SCI0569
MBOUMEU YOUMBI AXEL AUBIN	CM-UDS-21SCI0589
MOUNCHIKPOU MEFIRE HAMED KAMAL	CM-UDS-21SCI0601
NJIH ASONYU SENGE	CM-UDS-21SCI0625

- 1 Introduction
- 2 Random Forests
- 3 XGBoost
- 4 SVM
- 5 Conclusion Générale

- 1 Introduction
- 2 Random Forests
- 3 XGBoost
- 4 SVM
- 5 Conclusion Générale

Dans le domaine du machine learning supervisé, plusieurs algorithmes se distinguent par leur efficacité pour la classification et la régression. Parmi eux, les **SVM**, les **Random Forests** et **XGBoost** occupent une place centrale grâce à leur robustesse et leurs performances élevées sur des données complexes.

Ces modèles reposent sur des approches différentes :

- Les **SVM** maximisent la marge de séparation entre les classes ;
- Les **Random Forests** combinent plusieurs arbres de décision pour améliorer la généralisation ;
- **XGBoost** utilise le boosting pour corriger progressivement les erreurs.

- **Apprentissage automatique (Machine Learning)** : domaine de l'IA permettant aux systèmes d'apprendre à partir des données sans être explicitement programmés.
- **Classification** : tâche supervisée consistant à prédire une étiquette discrète (ex : spam / non-spam, malade / sain).
- **Régression** : tâche supervisée visant à prédire une valeur continue (ex : prix d'une maison, température).

Concepts clés : Bagging vs Boosting

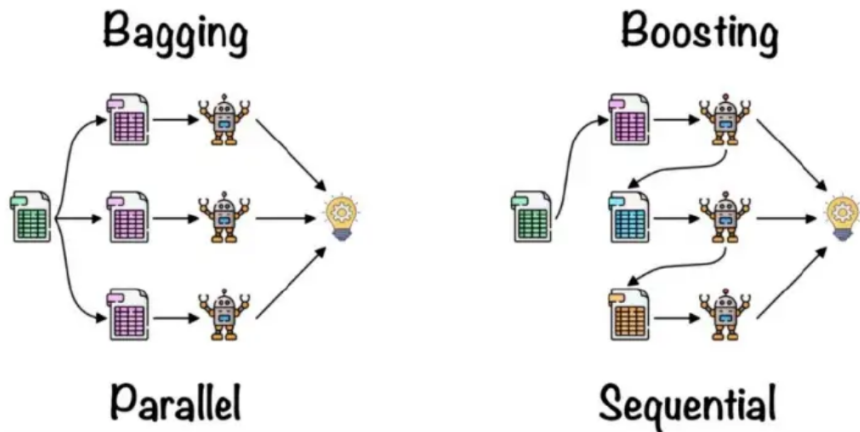


Figure 1 – Bagging (parallèle, réduit la variance) vs Boosting (séquentiel, réduit le biais).

- 1 Introduction
- 2 Random Forests
- 3 XGBoost
- 4 SVM
- 5 Conclusion Générale

- Les arbres de décision simples sont sujets au **surapprentissage (overfitting)**.
- Les **Random Forests** combinent plusieurs arbres pour un modèle plus **robuste et précis**.
- Principe clé : la *sagesse des foules* — agréger les prédictions de nombreux arbres entraînés sur des sous-échantillons différents.
- Contrairement aux SVM et à XGBoost, les Random Forests construisent des arbres **indépendants et parallèles**.

Fonctionnement du Random Forest

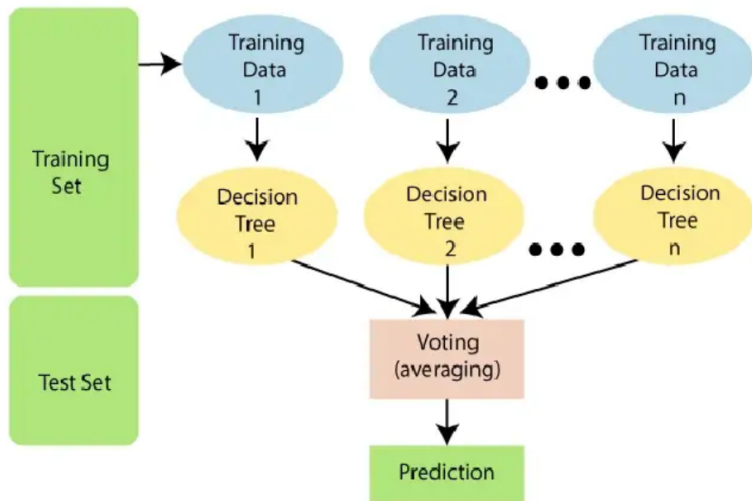


Figure 2 – Principe de fonctionnement du Random Forest

Rappel : Arbres de décision

- Un **arbre de décision** est un modèle hiérarchique basé sur une succession de questions binaires (oui/non).
- Il est composé de :
 - **Noeuds internes** : décisions sur les variables d'entrée ;
 - **Branches** : résultats possibles des décisions ;
 - **Feuilles** : prédictions finales.
- Exemple : « Âge > 30 ? », puis « Revenu > 50 000 ? » pour prédire un achat.

Rappel visuel : Arbre de décision

Exemple d'Arbre de Decision - Prediction d'achat

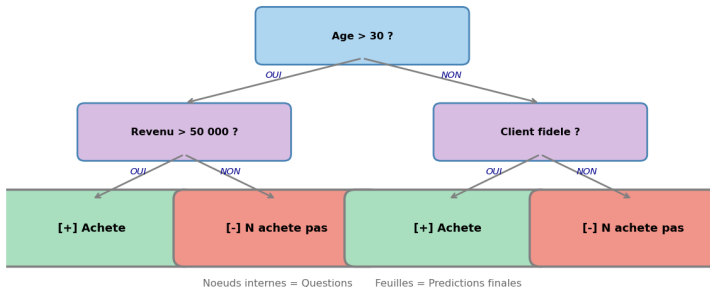


Figure 3 – Exemple d'arbre de décision pour la prédiction d'un achat.

- L'arbre se construit de manière **récursive** en choisissant à chaque étape la meilleure division.
- Sans contraintes, l'arbre peut **surapprendre** :
 - Excellente performance sur les données d'entraînement ;
 - Mauvaise généralisation sur de nouvelles données.
- Les arbres ont **faible biais** mais **forte variance** → motivation des Random Forests.

- Deux sources principales de hasard :
 - L'**échantillonnage bootstrap** des données ;
 - La **sélection aléatoire des caractéristiques**.
- Ces mécanismes permettent de construire un ensemble d'arbres **diversifiés et moins corrélés**.

Étape 1 : Échantillonnage Bootstrap

- Pour chaque arbre, on génère un échantillon aléatoire **avec remise**.
- Certaines observations peuvent apparaître plusieurs fois, d'autres peuvent être absentes.
- En moyenne, chaque échantillon contient environ **63,2%** d'observations uniques.
- Cela crée des jeux de données légèrement différents pour entraîner chaque arbre.

Étape 2 : Sélection aléatoire des caractéristiques

- À chaque nœud, on sélectionne aléatoirement un sous-ensemble de caractéristiques ($m < p$).
- La meilleure division est choisie **uniquement parmi ces caractéristiques**.
- Objectif : réduire la corrélation entre les arbres et augmenter la **diversité** du modèle.

Étapes 3 et 4 : Croissance et Agrégation

- **Croissance des arbres :**

- Chaque arbre est développé jusqu'à sa profondeur maximale (sans élagage).

- **Agrégation des prédictions :**

- Classification : **vote majoritaire** des arbres ;
- Régression : **moyenne** des prédictions.

Hyperparamètre clé : Nombre d'arbres

- **n_estimators (ntree)** : nombre d'arbres dans la forêt.
- Valeurs typiques : 100 à 1000 arbres.
- Effets :
 - Plus d'arbres $\rightarrow$ variance réduite et prédictions plus stables ;
 - Temps de calcul plus élevé.
- Les gains deviennent faibles après un certain seuil (plateau de performance).
- Contrairement au boosting, ajouter des arbres ne provoque généralement **pas de surapprentissage**.

Hyperparamètre clé : max_features

- Contrôle le nombre de caractéristiques testées à chaque division.
- Valeurs par défaut :
 - Classification : $\sqrt{p}$
 - Régression : $p/3$
- Effets :
 - Valeurs faibles $\rightarrow$ arbres plus diversifiés mais plus faibles ;
 - Valeurs élevées $\rightarrow$ arbres plus puissants mais plus corrélés.
- C'est l'hyperparamètre le **plus important** à ajuster.

- **Profondeur des arbres :**
 - `max_depth`, `min_samples_split`, `min_samples_leaf`
- **Taille bootstrap (`max_samples`) :**
 - Fraction des données utilisée pour chaque arbre.
- **Stratégie pratique :**
 - 1 Commencer avec les paramètres par défaut ;
 - 2 Ajuster d'abord `n_estimators` ;
 - 3 Ajuster ensuite `max_features` (le plus influent) ;
 - 4 Utiliser l'erreur OOB pour une évaluation rapide.

Estimation de l'erreur Out-of-Bag (OOB) — Illustration

Echantillonnage Bootstrap et données Out-of-Bag (OOB)

Dataset original (n=10 observations)



Arbre 1 (Bootstrap):



Arbre 2 (Bootstrap):



Arbre 3 (Bootstrap):



Bleu = utilise (bootstrap) Rouge/OOB = non vu = ~ 36.8% des données

L'erreur OOB est calculée sur ces obs. -> validation intégrée, sans jeu de test séparée !

Figure 4 – Schéma du mécanisme bootstrap et des données OOB par arbre.

Fonctionnement de l'erreur Out-of-Bag (OOB)

- Lors du bootstrap, environ **36,8%** des observations ne sont pas utilisées pour entraîner un arbre donné : ce sont les **échantillons Out-of-Bag (OOB)**.
- Pour chaque observation :
 - ➊ Identifier les arbres qui ne l'ont pas vue (arbres OOB).
 - ➋ Faire la prédiction uniquement avec ces arbres.
 - ➌ Comparer la prédiction à la vraie valeur.
- L'erreur OOB = moyenne des erreurs sur toutes les observations.
Avantage : validation intégrée sans jeu de test séparé.

Avantages :

- Estimation **non biaisée** de l'erreur de généralisation
- Aucun coût d'entraînement supplémentaire
- Permet de choisir le nombre d'arbres optimal

Limites :

- Peut être légèrement **optimiste** comparée à un test indépendant
- Moins fiable sur de **petits jeux de données**

Formule clé :

$$P(\text{non sélectionnée}) = \left(1 - \frac{1}{n}\right)^n \rightarrow \frac{1}{e} \approx 0,368$$

Importance des variables — Illustration

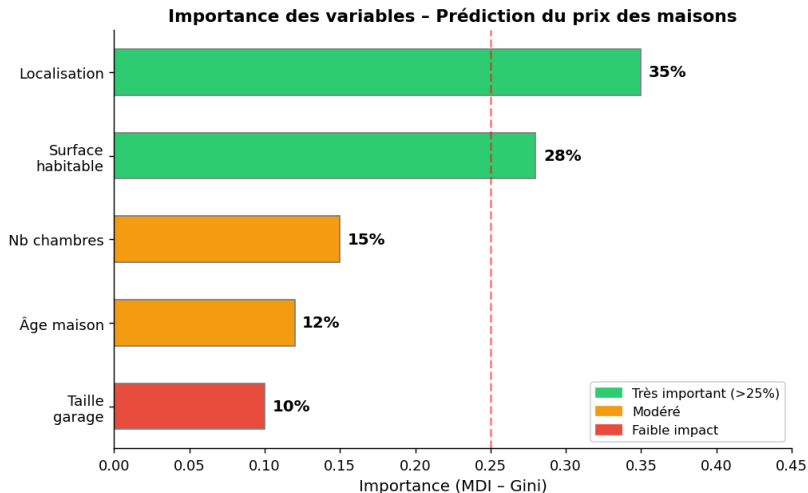


Figure 5 – Importance MDI (Gini) des variables pour la prédiction du prix des maisons.

Importance des variables : MDI (Gini Importance)

- La **Diminution Moyenne de l'Impureté (MDI)** mesure combien une variable réduit l'impureté des nœuds.
- Calcul : sommer la réduction d'impureté pondérée sur tous les arbres et normaliser.
- **Limites :**
 - Biais vers les variables à forte cardinalité
 - Peut surestimer l'importance si les variables sont corrélées

Avantages des Random Forests

- Robustesse au surapprentissage
- Prétraitement minimal des données
- Gestion des valeurs manquantes
- Capture naturelle des relations non linéaires
- Parallélisation et scalabilité
- Validation intégrée (erreur OOB)
- Importance des variables interprétable
- Bonnes performances avec paramètres par défaut
- Polyvalence (classification, régression, etc.)

- Difficulté d'interprétation
- Besoins élevés en mémoire
- Vitesse de prédiction plus lente
- Moins performant sur données hautement dimensionnelles
- Pas d'extrapolation en régression
- Importance des variables biaisée (MDI)
- Pas toujours le meilleur choix face au boosting ou deep learning

- Les Random Forests sont des modèles puissants et polyvalents basés sur le **bagging**
- Robustesse au surapprentissage et gestion des relations non linéaires
- Validation intégrée via l'**erreur Out-of-Bag (OOB)**
- Adaptées à la classification et à la régression
- Limites : interprétation difficile et coût mémoire élevé
- XGBoost adopte une approche **boosting** avec construction séquentielle des arbres

- 1 Introduction
- 2 Random Forests
- 3 XGBoost**
- 4 SVM
- 5 Conclusion Générale

- **XGBoost** (eXtreme Gradient Boosting) : algorithme avancé de gradient boosting
- Optimisé, scalable et performant pour classification et régression
- Introduit par Chen & Guestrin (2016)
- Large succès en compétitions Kaggle et applications industrielles
- Utilise le **boosting** (construction séquentielle) pour réduire le biais
- Alternative performante aux méthodes basées sur le bagging (Random Forest)

Principe du Gradient Boosting

- Apprentissage **itératif et séquentiel**
- Chaque nouvel arbre apprend les **erreurs résiduelles** du modèle précédent
- Construction progressive du modèle global
- Interprétation : descente de gradient dans l'espace des fonctions
- Objectif : minimisation incrémentale de la **fonction de perte**

Principe du Gradient Boosting — Illustration

Principe du Gradient Boosting - Apprentissage des résidus

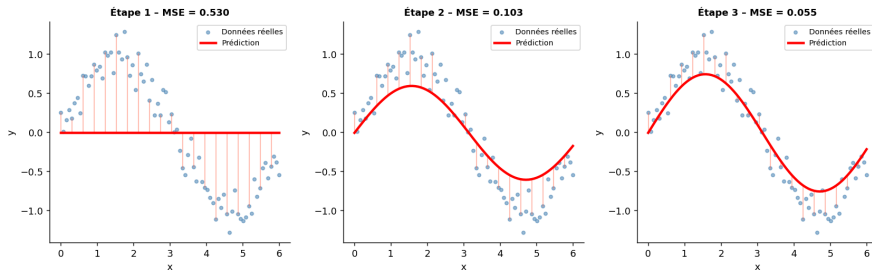
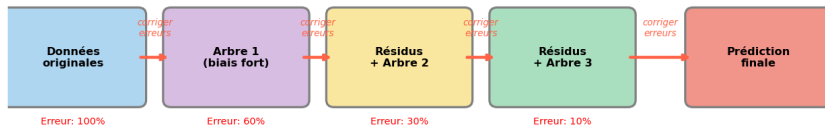


Figure 6 – Chaque arbre réduit progressivement les résidus du modèle précédent.

XGBoost : Construction séquentielle

XGBoost - Construction séquentielle par correction des erreurs



✂ Chaque arbre apprend sur les résidus du précédent → Réduction progressive du biais

Figure 7 – XGBoost corrige progressivement les erreurs des modèles précédents.

- Gestion native des valeurs manquantes (*Sparsity-aware split finding*)
- Gestion du déséquilibre de classes via `scale_pos_weight`
- Subsampling stochastique :
 - `subsample`
 - `colsample_bytree`, `colsample_bylevel`, `colsample_bynode`
- **Early stopping** pour déterminer automatiquement le nombre optimal d'arbres
- Régularisation avancée pour améliorer la généralisation

Subsampling stochastique — Illustration

Subsampling stochastique dans XGBoost (subsample + colsample_bytree)

Dataset complet (lignes = obs., colonnes = features)

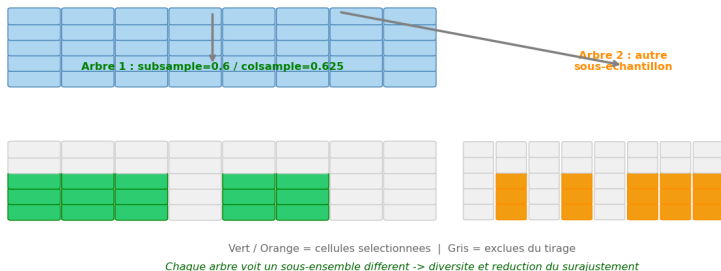


Figure 8 – Chaque arbre reçoit un sous-ensemble différent de lignes et colonnes.

Early Stopping — Illustration

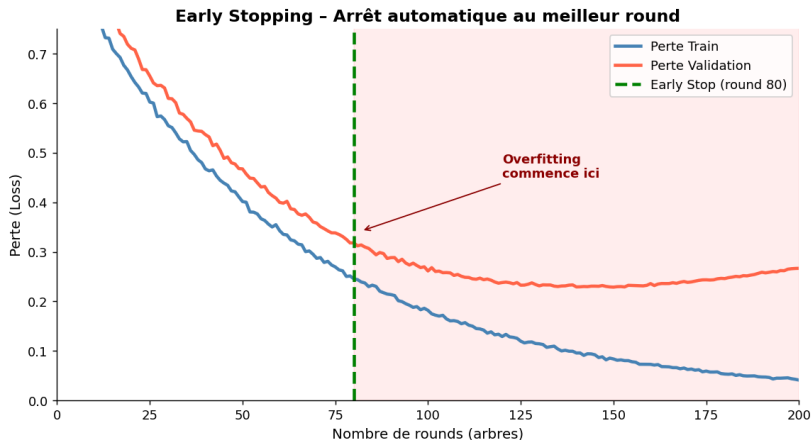


Figure 9 – L'entraînement s'arrête automatiquement au meilleur round de validation.

Régularisation XGBoost — Illustration

Effet de la régularisation XGBoost sur le surapprentissage

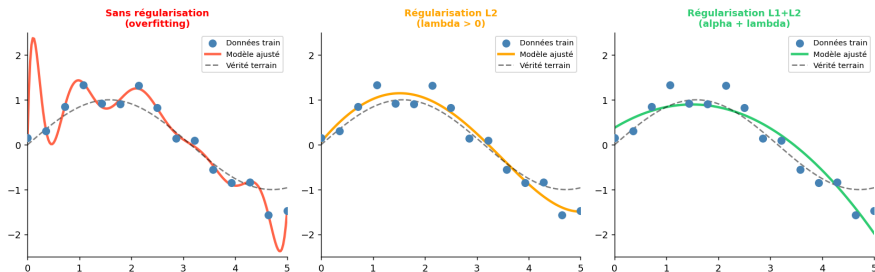


Figure 10 – La régularisation L1 (α) et L2 (λ) contrôle le surapprentissage.

Limitations du Gradient Boosting classique (GBM)

- Absence de régularisation explicite $\rightarrow$ risque élevé de surapprentissage
- Optimisation de premier ordre uniquement (gradient seul)
- Gestion limitée des valeurs manquantes
- Performances réduites sur grandes bases de données
- Manque d'optimisations systèmes pour le passage à l'échelle

- **Parallélisation** interne : recherche des meilleurs splits en parallèle
- Structure en colonnes (*Column Block*) pour accélérer les calculs
- Scalabilité sur grandes données :
 - **Histogram-based splits** (discrétisation en bins)
 - **Out-of-core computing** (gestion des données hors mémoire)
 - **Weighted Quantile Sketch** (approximation efficace des quantiles)

Histogram-based Split Finding — Illustration

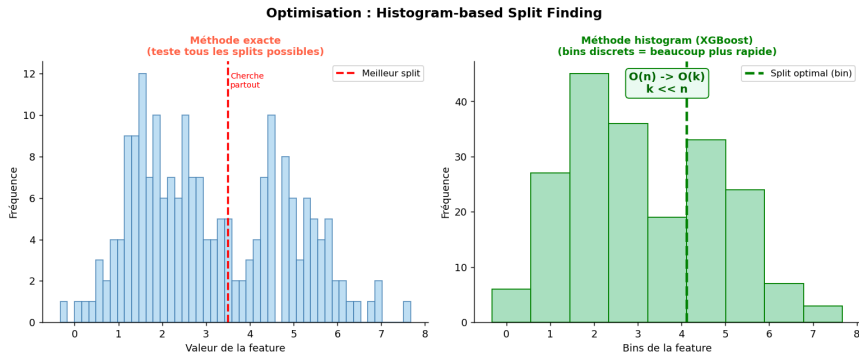
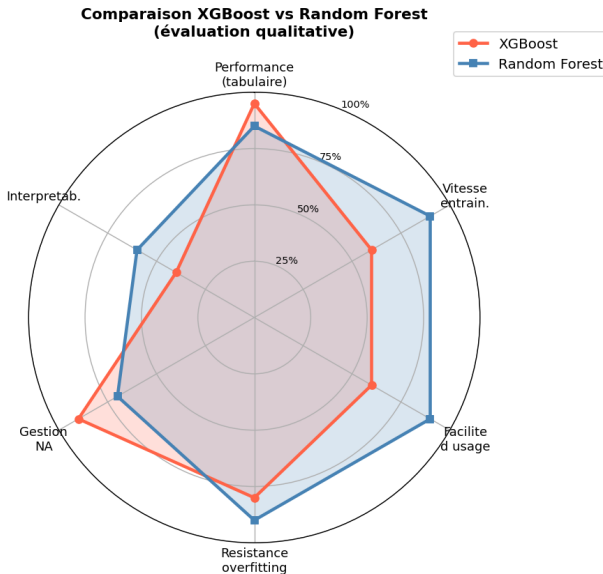


Figure 11 – XGBoost remplace la recherche exacte par des bins discrets : $O(n) \rightarrow O(k)$ avec $k \ll n$.

- **Contrôle de la complexité :**
 - `max_depth`, `min_child_weight`, `gamma`
- **Régularisation et apprentissage :**
 - `lambda` (L2) et `alpha` (L1)
 - `learning_rate`
- **Sampling et autres :**
 - `subsample`, `colsample_bytree`
 - `scale_pos_weight`
 - `n_estimators` + `early_stopping_rounds`

Comparaison XGBoost vs Random Forest — Vue Radar



- XGBoost représente l'**état de l'art** du gradient boosting
- Trois piliers : **précision**, **efficacité** et **praticité**
- Optimisation de second ordre régularisée
- Parallélisation et gestion mémoire optimisées
- Fonctionnalités natives avancées
- Choix privilégié lorsque la **performance prédictive est critique**
- Référence majeure malgré l'existence de LightGBM et CatBoost

- 1 Introduction
- 2 Random Forests
- 3 XGBoost
- 4 SVM**
- 5 Conclusion Générale

Définition

Algorithme d'apprentissage supervisé introduit par **Vapnik et al. (1992)** pour la **classification** et la **régression**.

Idée centrale :

- Trouver l'**hyperplan optimal** séparant les classes
- Maximiser la **marge** entre les deux classes
- Fondé sur l'**optimisation convexe**

Points forts :

- Robuste en grande dimension
- Résistant au surapprentissage
- Applicable aux données non linéaires

Support Vector Machines (SVM)

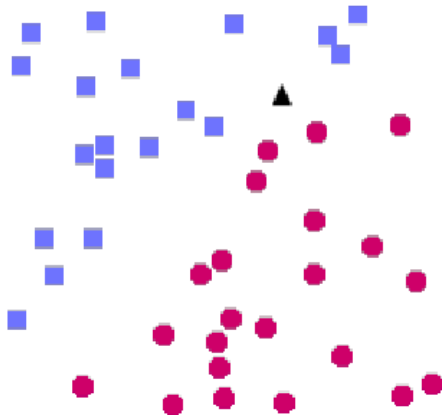


Figure 13 – L'objet à classer (le triangle noir) est-il un rond rouge ou bien un carré bleu ?

Support Vector Machines (SVM)

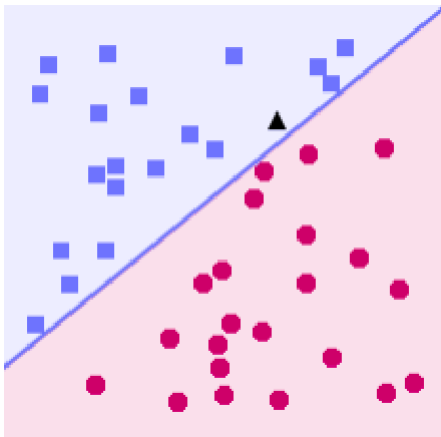


Figure 14 – Notre SVM, muni des données d'entraînement (les carrés bleus et les ronds rouges d' déjà indiqués comme tels par l'utilisateur), a tranché : le triangle noir est en fait un carre bleu

Pour $x \in \mathbb{R}^n$, le SVM cherche un **hyperplan affine** :

$$h(x) = \mathbf{w}^\top x + b$$

- $\mathbf{w}$: vecteur de poids b : biais
- Classification : $y = +1$ si $h(x) \geq 0$, $y = -1$ sinon

Contraintes d'apprentissage

$$y_i (\langle \mathbf{w}, x_i \rangle + b) \geq 1 \quad \forall i$$

Objectif : maximiser la marge

$$\text{Marge} = \frac{2}{\|\mathbf{w}\|} \quad \Rightarrow \quad \text{minimiser } \|\mathbf{w}\|$$

Maximisation de la marge

- H : hyperplan séparateur
- H^+ : hyperplan longeant les positifs
- H^- : hyperplan longeant les négatifs
- La **marge** = distance entre H^+ et H^-

Distance point-hyperplan

$$d = \frac{|\mathbf{w}^\top x_k + b|}{\|\mathbf{w}\|}$$

Les **vecteurs de support** sont les points les plus proches de H .

Résultat clé

$$\text{Largeur de la marge} = \frac{2}{\|\mathbf{w}\|}$$

$\Rightarrow$ maximiser la marge = minimiser $\|\mathbf{w}\|$

Maximisation de la marge

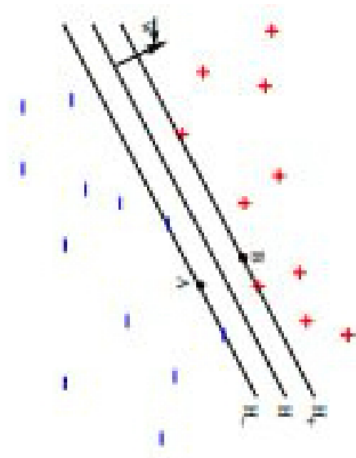


Figure 15 – Schéma illustratif : groupe H^+ et H^-

Formulation Lagrangienne des SVM

Problème primal : minimiser $\frac{1}{2}\|\mathbf{w}\|^2$ sous contraintes $y_i(\mathbf{w}^\top x_i + b) \geq 1$

On introduit les **multiplicateurs de Lagrange** $\alpha_i \geq 0$:

$$\mathcal{L}(\mathbf{w}, b, \alpha) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i [y_i(\mathbf{w}^\top x_i + b) - 1]$$

Conditions KKT :

$$\mathbf{w} = \sum_i \alpha_i y_i x_i, \quad \sum_i \alpha_i y_i = 0$$

$\Rightarrow$ **Problème dual** en α uniquement

Propriété clé

Seuls les points avec $\alpha_i > 0$ contribuent à $\mathbf{w}$: ce sont les **vecteurs de support**.

La fonction de décision devient :

$$h(x) = \sum_i \alpha_i y_i K(x_i, x) + b$$

SVM non linéaires : Plongée en dimension supérieure

Problème : Certaines données **ne sont pas linéairement séparables** dans l'espace d'origine (ex : cercles concentriques, problème XOR).

Solution : Projeter les données dans un espace de **dimension supérieure** E' via une fonction ϕ :

$$\phi : E \rightarrow E', \quad x \mapsto \phi(x)$$

- Dans E' , les données deviennent linéairement séparables
- Le SVM trouve un hyperplan dans E'
- Ce plan correspond à une **frontière non linéaire** dans E

Problème pratique

Plus la dimension de E' est grande, plus les calculs sont **coûteux**.

⇒ Le **Kernel Trick** résout ce problème !

SVM non linéaires : Plongée en dimension supérieure



Figure 16 – Données d'entraînement non linéairement séparables

SVM non linéaires : Plongée en dimension supérieure

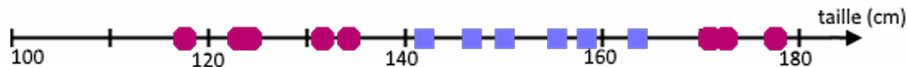


Figure 17 – Illustration : données sur une droite

SVM non linéaires : Plongée en dimension supérieure

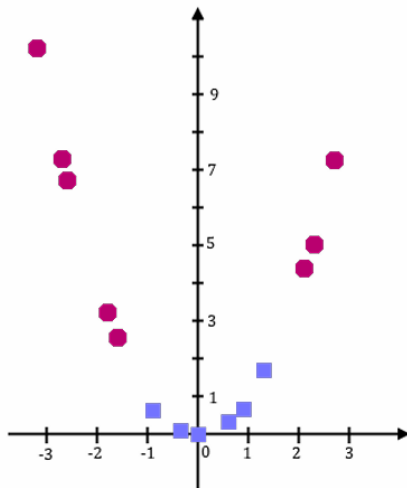


Figure 18 – Données d'entraînement après projection dans l'espace R^2

L'Astuce du Noyau (Kernel Trick)

Idée centrale : Les algorithmes n'ont besoin que des **produits scalaires** $\phi(x_i)^\top \phi(x_j)$.

On définit une **fonction noyau** : $K(x, x') = \phi(x)^\top \phi(x')$

Principaux noyaux :

- **Polynomial** : $K = (\alpha x^\top x' + \lambda)^d$
- **Gaussien (RBF)** :
$$K = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$$
- **Laplacien** : $K = \exp\left(-\frac{\|x - x'\|}{\sigma}\right)$
- **Rationnel quadratique**

Avantage

Calcul en $\mathcal{O}(n)$ au lieu de $\mathcal{O}(n^2)$ pour le produit dans E' .

Pas besoin de connaître E' ni ϕ explicitement !

Types de noyaux

Noyau	Formule	Usage
Linéaire	$x^\top x'$	Données séparables
Polynomial	$(\alpha x^\top x' + \lambda)^d$	Écriture, NLP
Gaussien (RBF)	$\exp\left(-\frac{\ x-x'\ ^2}{2\sigma^2}\right)$	Usage général
Laplacien	$\exp\left(-\frac{\ x-x'\ }{\sigma}\right)$	Bio-informatique
Rat. Quadratique	$1 - \frac{\ x-x'\ ^2}{\ x-x'\ ^2 + \sigma}$	Recommandation

Condition de Mercer

Une fonction K est un noyau valide si et seulement si elle est **continue, symétrique et semi-définie positive** :
$$\sum_{i,j} c_i c_j K(x_i, x_j) \geq 0$$

Hyperparamètres clés des SVM

Paramètre C (régularisation)

Contrôle le **compromis biais-variance** :

- C grand $\Rightarrow$ marge étroite, peu d'erreurs tolérées $\Rightarrow$ surapprentissage
- C petit $\Rightarrow$ marge large, plus d'erreurs tolérées $\Rightarrow$ sous-apprentissage

Paramètre σ / γ (noyau RBF)

Contrôle la **portée d'influence** de chaque point :

- σ petit $\Rightarrow$ frontière très locale
- σ grand $\Rightarrow$ frontière lissée

Paramètre d (noyau polynomial)

Degré du polynôme :

plus d est grand, plus la frontière est complexe.

Stratégie de réglage

Utiliser la **validation croisée** (grid search) pour trouver la combinaison (C, σ) optimale.

Mauvais choix $\Rightarrow$ dégradation sévère des performances.

Avantages

- Efficace en **grande dimension**
- Robuste au **surapprentissage**
- Fonctionne avec peu de données
- Frontières complexes via noyaux
- Optimum **global** garanti

Limites

- Sensible au choix des **hyperparamètres** (C , σ)
- Coûteux pour de **grands jeux** de données
- Choix du noyau non trivial
- Moins interprétable que les arbres

Message clé

Les SVM cherchent l'**hyperplan à marge maximale**. Pour les données non linéaires, le **Kernel Trick** permet de travailler implicitement dans un espace de grande dimension, sans en calculer explicitement les coordonnées.

- 1 Introduction
- 2 Random Forests
- 3 XGBoost
- 4 SVM
- 5 Conclusion Générale**

Conclusion Générale

SVM

- Hyperplan à **marge maximale**
- Kernel Trick pour données non linéaires
- Optimum **global** garanti
- Sensible aux hyperparamètres (C , σ)
- Coûteux sur grands jeux de données

Random Forest

- **Bagging** d'arbres indépendants
- Réduction de la **variance** via agrégation
- Validation intégrée (**OOB**)
- Peu sensible aux hyperparamètres
- Ne peut pas **extrapoler**

XGBoost

- **Boosting** séquentiel régularisé
- Approximation de **Taylor 2nd ordre**
- Gestion native des **valeurs manquantes**
- Meilleure performance sur données tabulaires
- Réglage plus complexe (+20 hyperparamètres)

Application : Détection du Diabète (PIDD)

Les trois algorithmes ont été évalués sur la **Pima Indians Diabetes Database** (768 patientes, 8 variables cliniques, 35% de cas positifs).

XGBoost obtient les meilleures performances globales (AUC-ROC, F1-score).

Random Forest excelle en robustesse et rappel.

SVM (RBF) est compétitif mais sensible au déséquilibre des classes.

Variable clé :

La **glycémie plasmatique** est systématiquement le prédicteur le plus informatif dans les trois modèles.

Merci pour votre attention !

Des questions ?